Orchestrazione
GameController
-lobbyManager: LobbyManager
-activePlayers: Map~String, Player~
-gameOrchestrators: Map~int, GameOrchestrator~
+main(args: String[]): void
+registerPlayer(player: Player): void
+unregisterPlayer(nickname: String): void
+isNicknameAvailable(nickname: String): bool
+createLobby(host: Player): Lobby
+joinLobby(lobbyId: int, player: Player): void
+exitLobby(lobbyId: int, player: Player): void
+startGameFromLobby(lobbyId: int): Game
-resolveGameOrchestrator(gameId: int): GameOrchestrator
-createGameOrchestrator(game: Game): GameOrchestrator
LobbyManager
-lobbies: Map~int, Lobby~
+createLobby(host: Player): Lobby
+joinLobby(lobbyId: int, player: Player): void
+removePlayer(lobbyId: int, player: Player): void
+canStart(lobbyId: int): bool
+createGameFromLobby(lobbyId: int): Game
+listOpenLobbies(): List~Lobby~
GameOrchestrator
-game: Game
-leaderboard: Leaderboard
+initializeGame(players: List~Player~): void
+playTurn(player: Player, space: OfferSpace): void
+finalizeRound(): void
+endMatch(): void
Lobby
-id: int
-hostNickname: String
-playerNicknames: List~String~
-status: LobbyStatus
-minPlayers: int = 2
-maxPlayers: int = 5
+addPlayer(player: Player): void
+removePlayer(player: Player): void
+canStart(): bool
+isFull(): bool
+getPlayers(): List~Player~
Gestione Stato
Game
-id: int
-playerCount: int
-players: Map~String, Player~
-currentEra: Era
-status: GameStatus
+setup(players: List~Player~): void
+playTurn(space: OfferSpace): void
+endRound(): void
+advanceEraIfNeeded(): bool
-initializeGameState(): void
-initializeTribeDeck(): void
-initializeBuildingDecks(): void
+calculateFinalScores(): Map~String, int~
+getPlayers(): List~Player~
+getStatus(): GameStatus
+isGameOver(): bool
+isRoundComplete(): bool
TurnOrderTrack
-currentPlayerIndex: int
+getCurrentPlayer(): Player
+advanceToNextPlayer(): void
+setPlayerOrder(players: List~Player~): void
+assignInitialOrder(players: List~Player~, startingTile: StartingTile): void
+rebuildOrderFromOfferPath(path: OfferPath, startingTile: StartingTile): void
TurnState
-currentRound: int
-maxRounds: int = 10
-playersCompletedTurn: int
-currentPhase: RoundPhase
-roundCompleted: bool
+advanceRound(): void
+markRoundCompleted(): void
+advancePhase(): void
+allPlayersCompleted(playerCount: int): bool
+isFinalRound(): bool
+clear()
Statistiche
Leaderboard
-pointsByNickname: Map~String, int~
+updateScores(scores: Map~String, int~): void
+getTotalPoints(nickname: String): int
+getScores(): Map~String, int~
Tavolo di Gioco
Board
+buildOfferPath(): void
+setupInitialRows(tribeDeck: TribeDeck, buildingDeck: BuildingDeck, playerCount: int): void
+discardLowerRow(): void
+shiftUpperToLower(): void
+refillUpperTribeCards(tribeDeck: TribeDeck, playerCount: int): void
+moveUpperBuildingCardsToLowerRow(): void
+refillUpperBuildingCards(buildingDeck: BuildingDeck): void
UpperPile
LowerPile
+discardLowerRow(): void
+resolveLowerRowEvents(players: List~Player~, tribeDeck: TribeDeck): void
+discardLowerBuildings(): void
OfferPath
+clearOfferSpaces(): void
+returnPlayersToStartingTile(): void
StartingTile
+initializeBlocks(): void
+resolveEffects(track: TurnOrderTrack): void
StartingBlock
-position: int
-foodModifier: int
-prestigeModifier: int
-occupiedBy: Player
+occupy(player: Player): void
+clear(): void
OfferSpace
-positionIndex: int
-code: String
-minPlayers: int
-foodBonus: int
-topRowCardsToTake: int
-bottomRowCardsToTake: int
-occupiedBy: Player
+isEnabledFor(playerCount: int): bool
+isAvailable(): bool
+occupy(player: Player): void
+resolveAction(game: Game, player: Player): void
+clear(): void
Players
Player
-nickname: String
-food: int
-prestigePoints: int
-totem: TotemColor
+takeCard(card: Card): void
+gainFood(amount: int): void
+payFood(amount: int): void
+gainPrestigePoints(amount: int): void
Tribe
Carte
Deck
+draw(): Card
+isEmpty(): bool
BuildingDeck
-era: Era
+initialize(): void
Card
+getEra(): Era
+isPlayableIn(playerCount: int): bool
TribeDeck
+initialize(): void
BuildingCard
-foodCost: int
-prestigePoints: int
-effect: String
CharacterCard
-characterType: CharacterType
-playerCountRequirement: int
EventCard
-playerCountRequirement: int
+applyEffect(player: Player, tribeDeck: TribeDeck): void
HuntEventCard
ShamanicRitualEventCard
CavePaintingsEventCard
SustenanceEventCard
Enums
CharacterType
HUNTER
SHAMAN
ARTIST
BUILDER
INVENTOR
GATHERER
InventionType
BOW
FIRE
WHEEL
KNIFE
RAFT
BASKET
SPEAR
TENT
NECKLACE
PAINTING
TotemColor
RED
BLUE
WHITE
YELLOW
BLACK
RoundPhase
PLAYER_TURN
LOWER_ROW_EVENTS
RETURN_TO_STARTING_TILE
STARTING_TILE_EFFECTS
ROW_CLEANUP
ROW_REFILL
ERA_ADVANCEMENT
Era
I
II
III
GameStatus
SETUP
IN_PROGRESS
ENDED
LobbyStatus
WAITING
READY
STARTED
CLOSED

Entry point per la pre-partita.
Mantiene il registro dei player attivi
indicizzati per nickname univoco.
Instrada ogni richiesta runtime
verso il GameOrchestrator corretto
usando il gameId della partita.
Crea e registra un GameOrchestrator
per ogni Game avviato.

Gestisce le lobby.
createGameFromLobby(...) valida la lobby,
recupera i player da Lobby.getPlayers()
e costruisce il Game corrispondente.

Wrapper applicativo della partita.
Tutte le richieste del controller
passano da qui prima di raggiungere Game.
Non contiene le regole del dominio,
ma coordina accesso, lifecycle
e salvataggio statistiche finali.

Temporarily modeled as enums.
Planned to evolve into subclasses.